Commons Guild Protocol (CGP): Tokenless, Forkable, Guild-Centric Communication with Verifiable Logs and Role-Based Relays

Ryan Hardesty Lewis
Institute for Data, Systems, & Society, MIT

***Abstract*—Modern real-time communities largely rely on centralized platforms, where a single operator controls governance, moderation, and data retention. Decentralized alternatives exist, but they typically optimize for different primitives—identity-centric feeds, global firehoses, or federated DAG state—which introduces a mismatch when building communities at scale. This paper introduces Commons Guild Protocol (CGP), a draft protocol for tokenless, forkable, guild-centric chat built on append-only signed logs inspired by hash chains and transparency systems. Commons Guild Protocol (CGP) deliberately avoids mandatory proof-of-work, mining, or per-message fees, and instead supports *optional relay plugins* (e.g., anti-spam puzzles, proof-of-compression, media pinning, or on-chain anchoring). A key architectural choice is a *role-based relay architecture* with separable discovery and data roles: (i) independent *directory operators* act in a discovery role, maintaining transparency logs that map human-readable handles to guild identifiers and relay endpoints; and (ii) separate *guild relays* sequence and serve each guild's event log, with additional relay roles as needed. We analyze trade-offs relative to Matrix, Secure Scuttlebutt (SSB), AT Protocol/Bluesky, Nostr, and Push Protocol, and outline extensions for verifiable checkpointing, scalable media handling, and optional proofs.**

***Index Terms*—decentralized communication, transparency logs, Merkle trees, append-only logs, governance, forkability, guild chat, content addressing**

I. Introduction

Centralized chat systems deliver excellent user experience but concentrate control over identity, moderation, and history. Discord popularized *guilds* (servers), *channels*, and *roles* as the primary mental model, yet its data plane and governance ultimately depend on a single operator [1].

Decentralized protocols span a wide design space. Matrix federates rooms across independently operated servers and resolves shared room state over an event DAG [2]. Secure Scuttlebutt (SSB) replicates identity-centric append-only feeds peer-to-peer [3]. The AT Protocol separates personal data servers (PDS), relays, and app views for social applications [4]. Nostr publishes signed events to relays with optional conventions [5]. Push Protocol roots messaging in Web3 primitives and chain-centric assumptions [6]. Yet each of these systems is tuned to a different core primitive (global feeds, social graphs, or chain-centric messaging).

This paper introduces Commons Guild Protocol (CGP), a *tokenless* and *PoW-optional* group chat protocol that borrows Bitcoin-inspired tamper-evident data structures (hash chains and Merkle roots) [7], [8]. Commons Guild Protocol (CGP) keeps message payloads off-chain while allowing optional anchoring for auditability. The base standard is intentionally minimal: it mandates only core integrity invariants and wire compatibility, while relay operators can layer additional policies—such as ephemerality, anti-spam proofs, compression budgets, or media pinning—without fragmenting interoperability.

II. Design Goals and Non-goals

A. Goals

G1: Guild-first model. Guilds, channels, and roles are first-class, so clients get server/room semantics directly instead of encoding them into per-user feeds or DAGs.

G2: Tamper-evident history without blockchain dependence. A guild is an append-only signed event log with hash-linked structure. Clients can verify integrity locally without requiring a blockchain.

G3: Forkable governance. Communities can fork a guild's history at a known checkpoint and continue under new rules or operators (analogous to software forks, inspired by the social dynamics of ledger forks [7]).

G4: Transparent, role-based discovery. Usernames resolve through directory operators that maintain Merkle-based transparency logs (in the spirit of Certificate Transparency [9]) and are cleanly separated from guild relays.

G5: Thin clients and selective sync. Browsers and mobile clients should sync only the guilds and channels they care about, with configurable retention windows.

B. Non-goals

N1: Global consensus. Commons Guild Protocol (CGP) does not attempt to maintain a single canonical world state, unlike blockchains.

N2: Exactly-once delivery. The protocol targets eventual consistency with at least one honest relay.

N3: Mandatory end-to-end encryption. Payload encryption can be layered on top (e.g., using Signal-style primitives [10]), but it is not required by the format.

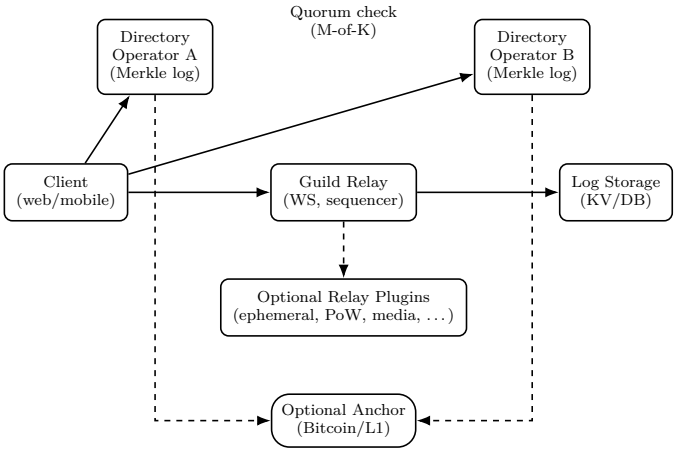


Fig. 1. System overview. Directory operators provide transparency-backed discovery; guild relays sequence and serve logs; optional anchoring and relay plugins add policy without breaking core interoperability.

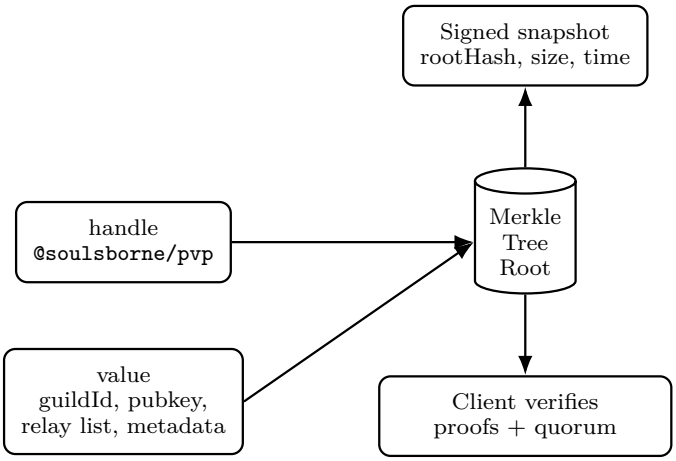


Fig. 2. Directory operators commit mappings into Merkle trees and sign snapshots, enabling inclusion and consistency proofs.

III. Architecture Overview

A. Role-based relay architecture

Commons Guild Protocol (CGP) organizes the network into orthogonal *relay roles*, as seen in Figure 1. In the simplest deployment we highlight two:

(1) a **discovery role**, in which multiple directory operators maintain *transparency logs* that map a handle (e.g., `@soulsborne/pvp`) to a guild identifier, public key, and relay descriptors

(2) a **guild role**, in which one or more guild relays store and serve the guild log and provide sequencing for events.

Additional relay roles (e.g., media storage, anti-abuse, or archival relays) can be added without changing the core log format. This role-based separation decouples *finding* a guild from *hosting* its history, and avoids forcing a single global feed-aggregator architecture (e.g., relay-driven "firehose" distribution in AT-Protocol-style systems [4]) for chat-style selective participation.

B. Guild log as a verifiable append-only structure

A guild is defined by its genesis event. Subsequent events increment a sequence number and include the previous event hash, forming a hash chain [7]. Each event is signed using secp256k1 parameters standardized by SEC 2 [11], with optional Schnorr signatures [12].

C. Directory transparency logs

Directory operators provide *inclusion proofs* and *consistency proofs* (Figure 2), mirroring transparency-log approaches [9]. Clients consult multiple operators and require a quorum agreement to reduce reliance on any single naming authority. Directory operators become auditable log witnesses rather than a single trusted registrar.

IV. Core: Events, Verification, and Sync

A. Wire model

The reference transport is WebSocket [13], using simple JSON array frames, similar in spirit to Nostr's relay framing [5]. Relays advertise optional `features` during handshake, allowing clients to adapt without hard-forking the protocol.

B. Event verification

Clients and relays reject events that violate the following conditions: monotonic `seq`; correct `prevHash`; valid `id = SHA256(canonical(event))`; and a valid signature under the author key (secp256k1 [11]).

C. Computing channel state

Channel state is derived deterministically by replaying the log. A `MESSAGE` event creates a message; `DELETE` removes it; and `EDIT` updates the visible representation. This yields a simple "UTXO-like" model (create/spend/update) without requiring full DAG state resolution.

V. Governance and Forkability

A core differentiator of Commons Guild Protocol (CGP) is explicit, protocol-level support for *forking*. A fork event references a parent guild, a parent sequence number, and the parent root hash, enabling clients to render lineage and verify the fork point.

Forkability reframes moderation disputes as *exit rights*: communities can migrate to new relays or new rule sets while keeping cryptographic continuity with the original history. In practice, this lets users keep their history without being locked into any one operator's governance.

VI. Comparative Analysis

Table I summarizes key trade-offs compared with widely deployed decentralized protocols.

TABLE I
QUALITATIVE COMPARISON: COMMONS GUILD PROTOCOL (CGP) VERSUS POPULAR DECENTRALIZED ARCHITECTURES.

System	Primary primitive	Conflict handling	Discovery / names	Fit for guild-style group chat
Commons Guild Protocol (CGP) (this work)	Guild-scoped append-only log; sequenced by relay	Single total order per guild relay; explicit forks	Transparency directories with quorum proofs (CT-like) [9]	Native: guilds / channels / roles are first-class; selective sync
Matrix [2]	Federated room event DAG plus state events	State resolution over DAG; no global finality [2]	Homeserver namespaces; federation [2]	Powerful but complex state resolution; guild semantics are application-layer
Secure Scuttlebutt (SSB) [3]	Identity centric append only feeds replicated peer-to-peer	Local subjectivity; replication by social graph	No global namespace by design; local introduction models	Feeds map poorly to server or role semantics without indexing
Bluesky/AT Protocol [4]	User repositories (content-addressed); PDS plus relays and app views	User repo commits; app-level views; aggregation via relay streams [4]	DNS-based handles bound to DIDs [4]	Strong for feeds and social graphs; chat needs a new model and selective sync
Nostr [5]	Signed events plus relays	Client chooses relays; no built-in global ordering [5]	Optional conventions for names and verification; no core namespace [5]	Minimal core is good for broadcast; governance and chat semantics are non-native
Push Protocol [6]	Web3-native messaging and notifications	Chain-centric assumptions; app-level semantics	Web3 identities and wallets	Useful for Web3 chat but token/chain coupling often misaligns with governance

A. Why Commons Guild Protocol (CGP)'s model can be preferable for guild chat

Reducing state complexity. Matrix's federation and state resolution are designed to tolerate concurrent event graphs across homeservers [14]. For guild-style chat, where users expect a clean linear timeline per channel, Commons Guild Protocol (CGP)'s relay-assigned sequencing offers a simpler, deterministic replay model at the cost of relying on at least one reachable sequencer per guild.

Separating discovery from hosting. AT Protocol's architecture separates hosting (PDS) from aggregation (relays) and views (app views) [15], [16]. Commons Guild Protocol (CGP) similarly separates planes, but aligns them with *guild discovery* (directory transparency) and *guild data* (log relays). This enables communities to change relay operators without renaming the guild and to detect naming rewrites via transparency proofs [9].

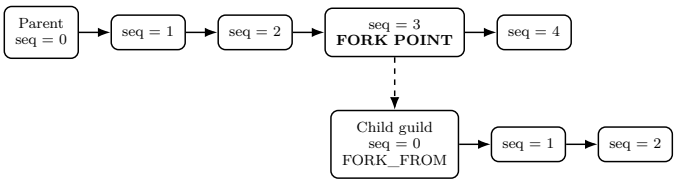


Fig. 3. Forking: a new guild log can branch from a parent at a verifiable checkpoint. Clients can present forks as alternative histories with shared provenance rather than hidden conflicts.

Explicit exit via forks. While forks exist socially in many ecosystems, Commons Guild Protocol (CGP) makes forking a first-class governance action encoded in the log. This makes community splits legible to clients, rather than being hidden in out-of-band coordination (Figure 3).

B. Relation to neighboring designs.

Relation to Git-style DVCS and hosting. A Commons Guild Protocol (CGP) guild log resembles a Git repository: both are append-only histories with signed updates and explicit forks, and both can be hosted by third parties. Git hosting platforms (e.g., GitHub) bundle naming, hosting, and access control into a single service. Commons Guild Protocol (CGP) instead splits these concerns across relay roles: directory operators implement a transparent naming and discovery role, guild relays sequence chat events, and optional media or policy relays. This unbundling lets communities move or replicate each role independently while keeping chat and governance semantics tailored to guilds rather than code.

Relation to Nostr and NIPs. Nostr standardizes a generic event + relay model and leaves richer semantics to optional Nostr Improvement Protocols (NIPs) [17]. Commons Guild Protocol (CGP), by contrast, depends on per-guild, hash-linked logs, CT-style directory transparency, and explicit relay roles. While these could be approximated as a Nostr profile, they require CGP-specific relay behavior and rules that go beyond Nostr's "dumb relay" philosophy, so we specify Commons Guild Protocol (CGP) as a separate guild-centric protocol rather than as a NIP.

VII. Massive Data and Rich Media (MP4, Attachments, Archives)

Large binaries stress any messaging system. Centralized platforms typically enforce upload caps and rely on HTTP caching / edge delivery infrastructure for attachments [18]. Matrix places storage and serving responsibilities on homeserver operators as part of its federated design [2]. The AT Protocol separates signed records from content-addressed blobs, keeping large media out of the repository history [4]. In Commons Guild Protocol (CGP), we follow this separation-of-concerns approach explicitly, treating rich media as a distinct plane that can scale and evolve independently of the guild log.

A. CGP approach: keep the log small and media pluggable

We recommend treating media as separate *blob objects* referenced from log events by content hash plus metadata (MIME type, size, and optional thumbnails). This aligns with content-addressed storage and deduplication principles used in systems such as IPFS [19] and AT Protocol [20]. We couple this system with optional relay plugins and show the pipeline in Figure 4.

B. Scaling considerations

Streaming video. For MP4 and long-form video, Commons Guild Protocol (CGP) clients can prefer manifests (HLS/DASH) represented as blob sets, with the log referencing the manifest root hash and track hashes. This prevents "messages as megabytes" attacks while enabling partial retrieval.

Retention and cost. Centralized platforms often monetize higher resource limits via subscription tiers (e.g., Discord's Nitro/boost model) [21]. Commons Guild Protocol (CGP) can support similar cost controls *without* central ownership by letting relays declare and enforce policies: (1) maximum blob size per guild; (2) time-to-live (TTL) for unpinned blobs; and (3) optional pay-by-resource plugins, while keeping the base standard tokenless.

Archival exports. Logs can be exported as compact, verified sequences. Blobs can be exported as CAR-like bundles (as in AT Protocol repositories [22]) or as content-addressed directories (IPFS-style) [19].

VIII. Security and Trust Model

A. Tamper evidence and equivocation

Hash-linked logs make rewrites detectable once a client has seen any checkpoint. However, a malicious relay could *equivocate* by serving different log heads to different clients. Commons Guild Protocol (CGP) mitigates this with: (i) client cross-checking across relays (multiple relays per guild), (ii) explicit checkpoints that are signed and optionally anchored, and (iii) directory transparency that can reveal inconsistent claims using consistency proofs [9].

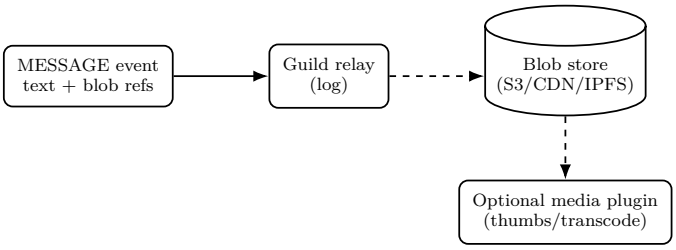


Fig. 4. Recommended media plane. Logs remain compact; events reference blobs by hash or CID; optional plugins add thumbnails, transcoding, or pinning policies.

B. Optional anchoring: checkpoints on-chain

Commons Guild Protocol (CGP) does not require a blockchain but can harden history by anchoring (a) directory Merkle roots and/or (b) guild log root hashes. A practical method is to commit small hashes into Bitcoin transactions using null-data outputs (OP_RETURN) [7]. Anchoring can be implemented by periodically publishing a checkpoint commitment into a public blockchain transaction, using Merkle batching to amortize cost across many checkpoints [7], [8], [23].

C. Spam, Sybils, and abuse

Rather than mandating a universal mechanism, Commons Guild Protocol (CGP) treats anti-abuse as a *relay policy plugin surface*. Options include: proof-of-work gating (e.g., Hashcash-style puzzles) [24]; rate limits (e.g., HTTP 429 "Too Many Requests") [25]; membership proofs (invite codes, WebAuthn, or reputation signals); and bandwidth or compression budgets, as discussed below.

IX. Future Work

A. Proof-of-work (optional, per relay)

Proof-of-work (PoW) can deter spam in open relays without requiring tokens. A relay could require a lightweight Hashcash token for `PUBLISH` requests in high-risk channels [24]. Importantly, this remains configurable: small private guilds can disable PoW entirely, and public megaguilds can enable it only for new accounts or suspicious bursts.

B. Proof-of-compression and "bandwidth fairness"

A distinctive extension is *proof-of-compression*: rather than charging money, a relay can require that large payloads (e.g., pasted logs, images, short clips) be accompanied by: (1) deterministic compression parameters (codec and level); (2) a commitment to the compressed bytes (hash or CID); and (3) a verifier rule that checks the compressed artifact meets target size bounds. This creates a "cost" denominated in user time and compute rather than currency. For more cryptographic rigor, one can explore succinct proofs and commitment schemes, but a practical first step is deterministic compression plus hash commitments and sampling audits.

C. Proof-of-retention for relays (auditable storage)

Large guilds will want assurances about what relays actually retain. Proofs of Retrievability (PoR) and Provable Data Possession (PDP) provide foundations for verifying that an untrusted storage provider still holds content [26], [27]. A Commons Guild Protocol (CGP) "retention plugin" could challenge relays to produce probabilistic proofs for log ranges and pinned blobs, creating a verifiable service-level agreement (SLA) without requiring a blockchain.

D. Checkpointing conversation histories on-chain

We envision two complementary checkpointing strategies: (i) directory-root anchoring for handle integrity via periodic Merkle roots [9]; and (ii) guild-root anchoring for chat history via periodic log-head hashes. Anchoring can be done directly via Bitcoin null-data outputs [28] or via timestamping systems (e.g., OpenTimestamps batching) [29], [30]. Clients can then show "verified as of block height X" for sensitive governance events (bans, elections, forks).

E. Media sharding and interoperability with CDNs

Commercial chat systems typically centralize attachment hosting and enforce plan- or policy-based limits. Commons Guild Protocol (CGP) can replicate similar UX while remaining protocol-neutral by standardizing: (1) a blob-reference schema (hash plus metadata); (2) an optional "media relay" role; and (3) transform attestations (thumbnail/transcode hashes). This mirrors AT Protocol's explicit separation of records from blobs [20], [22] while allowing alternate backends such as S3-compatible stores, IPFS pinsets [19], or community-operated CDNs.

X. Conclusion

Commons Guild Protocol (CGP) targets a pragmatic point in the design space: guild-style group chat with verifiable history and forkable governance, without imposing tokens, mining, or mandatory global consensus. By combining append-only signed logs with transparency-based discovery and a role-based relay architecture, Commons Guild Protocol (CGP) provides a foundation for interoperable UX while keeping policy extensible through optional relay roles and plugins. Future work can add selective proofs (PoW, compression, retention), scalable media planes, and optional on-chain checkpointing for high-integrity communities.

References

[1] "Guild resource | discord developer documentation," https://discord.com/developers/docs/resources/guild.

[2] F. Jacob, D. Becker, M. Grashöfer, and H. Hartenstein, "Matrix decomposition: Analysis of an access control approach on transaction-based dags without finality," in *Proceedings of the 25th ACM Symposium on Access Control Models and Technologies (SACMAT '20)*, 2020.

[3] D. Tarr, E. Lavoie, A. Meyer, and C. Tschudin, "Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications," in *Proceedings of the 6th ACM Conference on Information-Centric Networking (ICN '19)*, 2019. [Online]. Available: https://conferences.sigcomm.org/acm-icn/2019/proceedings/icn19-19.pdf

[4] M. Kleppmann, "Bluesky and the AT protocol: Usable decentralized social media," 2024.

[5] Z. Wei and G. Tyson, "Exploring the nostr ecosystem: A study of decentralization and resilience," 2024.

[6] "Push chain litepaper," https://push.org/litepaper.pdf.

[7] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," https://bitcoin.org/bitcoin.pdf, 2008.

[8] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology — CRYPTO '87*, 1987.

[9] B. Laurie, A. Langley, and E. Kasper, "Rfc 6962: Certificate transparency," https://datatracker.ietf.org/doc/html/rfc6962, 2013.

[10] T. Perrin and M. Marlinspike, "The double ratchet algorithm," https://signal.org/docs/specifications/doubleratchet/doubleratchet.pdf.

[11] Standards for Efficient Cryptography Group (SECG), "SEC 2: Recommended Elliptic Curve Domain Parameters (Version 1.0)," https://www.secg.org/SEC2-Ver-1.0.pdf, 2000.

[12] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, 1991.

[13] I. Fette and A. Melnikov, "Rfc 6455: The websocket protocol," https://datatracker.ietf.org/doc/html/rfc6455, 2011.

[14] "Room version 2 | matrix specification," https://spec.matrix.org/v1.12/rooms/v2/.

[15] "Federation architecture | bluesky documentation," https://docs.bsky.app/docs/advanced-guides/federation-architecture.

[16] "Sync | at protocol specification," https://atproto.com/specs/sync.

[17] "Nip-01: Basic protocol flow (nostr)," https://github.com/nostr-protocol/nips/blob/master/01.md.

[18] R. Fielding, M. Nottingham, and J. Reschke, "Rfc 7234: Hypertext transfer protocol (http/1.1): Caching," https://datatracker.ietf.org/doc/html/rfc7234, 2014.

[19] J. Benet, "IPFS — content addressed, versioned, P2P file system," https://arxiv.org/abs/1407.3561, 2014.

[20] "Blobs | at protocol specification," https://atproto.com/specs/blob.

[21] K. Robinson, "Yeet nitro boosted: A postdigital perspective on young people's literacy engagements with discord platform," *Literacy Research: Theory, Method, and Practice*, vol. 71, no. 1, pp. 359–376, 2022.

[22] "Repository | at protocol specification," https://atproto.com/specs/repository.

[23] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.

[24] A. Back, "Hashcash: A denial of service counter-measure," https://www.hashcash.org/hashcash.pdf, 2002.

[25] M. Nottingham and R. Fielding, "Rfc 6585: Additional http status codes," https://datatracker.ietf.org/doc/html/rfc6585, 2012.

[26] A. Juels and B. S. K. Jr., "Pors: Proofs of retrievability for large files," in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS '07)*, 2007.

[27] G. Ateniese, R. Burns, R. Curtmola, J. Herring, L. Kissner, Z. Peterson, and D. Song, "Provable data possession at untrusted stores," in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS '07)*, 2007.

[28] "Transactions | bitcoin developer guide," https://developer.bitcoin.org/devguide/transactions.html.

[29] "Opentimestamps (project site)," https://opentimestamps.org/.

[30] P. Todd, "Opentimestamps: Scalable, trust-minimized, distributed timestamping with bitcoin," https://petertodd.org/2016/opentimestamps-announcement, 2016.